The Publishing Accelerator Toolkit (PacTk)

Amy Neustein, Ph.D.
CEO and Founder
Linguistic Technology Systems
Fort Lee, NJ
917-817-2184
amy.neustein@verizon.net

"Scientific Publishing is a Scientific Discipline"

Class Libraries and Full-Text Search/Encoding Tools for Publishing Documents, Datasets, and Multimedia Resources

Overview ▸▸▸

In contemporary publishing, the standard workflow includes **XML**-based formats such as **JATS** or **BITS**, which become the basis for **PDF**s and other human-visible documents. However, **XML** itself is not explicitly built for representing textual data. As a result, examining **JATS** or **BITS** files via generic **XML** technology renders certain publishing-related query and editing tasks difficult to implement. The **PDF** format is similarly opaque because internal **PDF** representations of document text include many anomalies due to ligatures, hyphenation, font specs, and other layout-related discrepancies between **XML** inputs and user-visible output (in short, internal **PDF** text is only a partial representation of textual content as understood by authors and editors). As a consequence, **PDF** files are opaque to full-text search languages such as **XQFT** (**XQuery** Full Text), **DSL** (Dimensions Search Language), and OpenSearch Query **DSL** (Domain-Specific Language).

Given the limitations of **PDF** and **XML** for natural-language documents, numerous competing formats how possible applications within publishing workflows such as **TEI** (Text Encoding Initiative), **TAGML** (Text-as-Graph Markup Language), **BioC** (a biomedical text mining standard defined by PubMed Central), and Markdown (popular for wikis and social media). The **PacTk** system presents a different kind of alternative, based on a class library implemented in computer code rather than any single markup language.

PacTk's design reflects trends in scientific research and the emergence of interdisciplinary fields such as social ecology, translational medicine, and medical humanities. It is the view of **PacTk**'s developers that the value of new investigations often cannot be fully expressed in single-disciplinary terms; nor does one individual publication mark the end of a well-conceived research project. Instead, any single publications are milestones in the unfolding of projects that, in the best case, progress to socially-beneficial deployments. More so than in the past, research projects (summarized but not fully articulated by academic papers) can serve as foundations for scholars' and scientists' technical "identity," potentially more significant than details such as degree specialization or disciplinary affiliation. In recognition of these paradigms, **PacTk** seeks to help authors build multi-faceted presentations covering data, code, and foundations for practical applications, which authors could build upon for future research work, funding, translational implementation, and then implementational feedback.

Examples ▸▸▸

PacTk represents text internally via a "Topomorphic Sentence Object Model" (**TSOM**), wherein sentences and other discursively signifant text areas are instances of **C++** classes, instead of relying on markup to organize documents. **PacTk** provides a format called **GTagML** to initialize **TSOM** data, but **XML** or **TAGML** input sources could be used as well. The following cases may help to clarify why an object-based intermediate representation is more efficient than working with markup directly.

- **Compiling a Meta-Index** **PacTk** helps book series, science grids, and potentially individual journals merge indexes from multiple documents (wherever those indexes exist in the first place) into a full-text database augmented with index metadata. This is distinct from a "reverse index" typically employed by search engines, which is based solely on the words in each document. A Meta-Index, by contrast, incorporates index entries, subheadings, "see also" redirection, and content locators that have been vetted by authors or editors, to confirm that a particular sentence or paragraph is a correct match to an indexed concept. Indexing software based on **TSOM** can be embedded into **PDF** viewers, allowing readers to access Meta-Index search portals directly from **PDF** files via context menus (Figure 3 and 6, 7, 8,10). In addition, match results (both from local and meta indexes) can be presented to users via navigable controls superimposed alongside the main text (Figure 4). **PacTk** can also help with **GUI** components for authors and editors to construct indexes to begin with (Figures 1, 2, 5, 9). Via **TSOM**, index entries are structured objects that can be merged into a full-text database while also generating the print index at the end of a book/publication. Furthermore, the search periphery can be expanded to include computer code, Research Object data sets, and other supplemental materials packaged with a publication.

  As a related use case, consider how comment boxes might be leveraged to construct a printed index (or also, secondarily, a search-engine/vector-database sort of index) with attention to rhetorical details. When the text of a manuscript provides a citation or names a person or theory, authors could be encouraged — either directly through input sources or via **PDF** annotations — to summarize positions taken vis-à-vis that subject matter (e.g., endorse, critique, or neutral/evaluative). When the sources or comments are analyzed to extract those notations, the printed index could be extended by incorporating that material into index entries. For example, instead of an undifferentiated heading locating "Keynesian economics," the index could produce subheadings such as "Keynesian economics, arguments for"; "Keynesian economics, arguments against"; "Keynesian economics, mathematical models"; "Keynesian economics, and macroeconomic theory" and etc. Via these annotations, Meta-Index search can hone in on articles that are most directly relevant from a rhetorical/argumentational point of view.

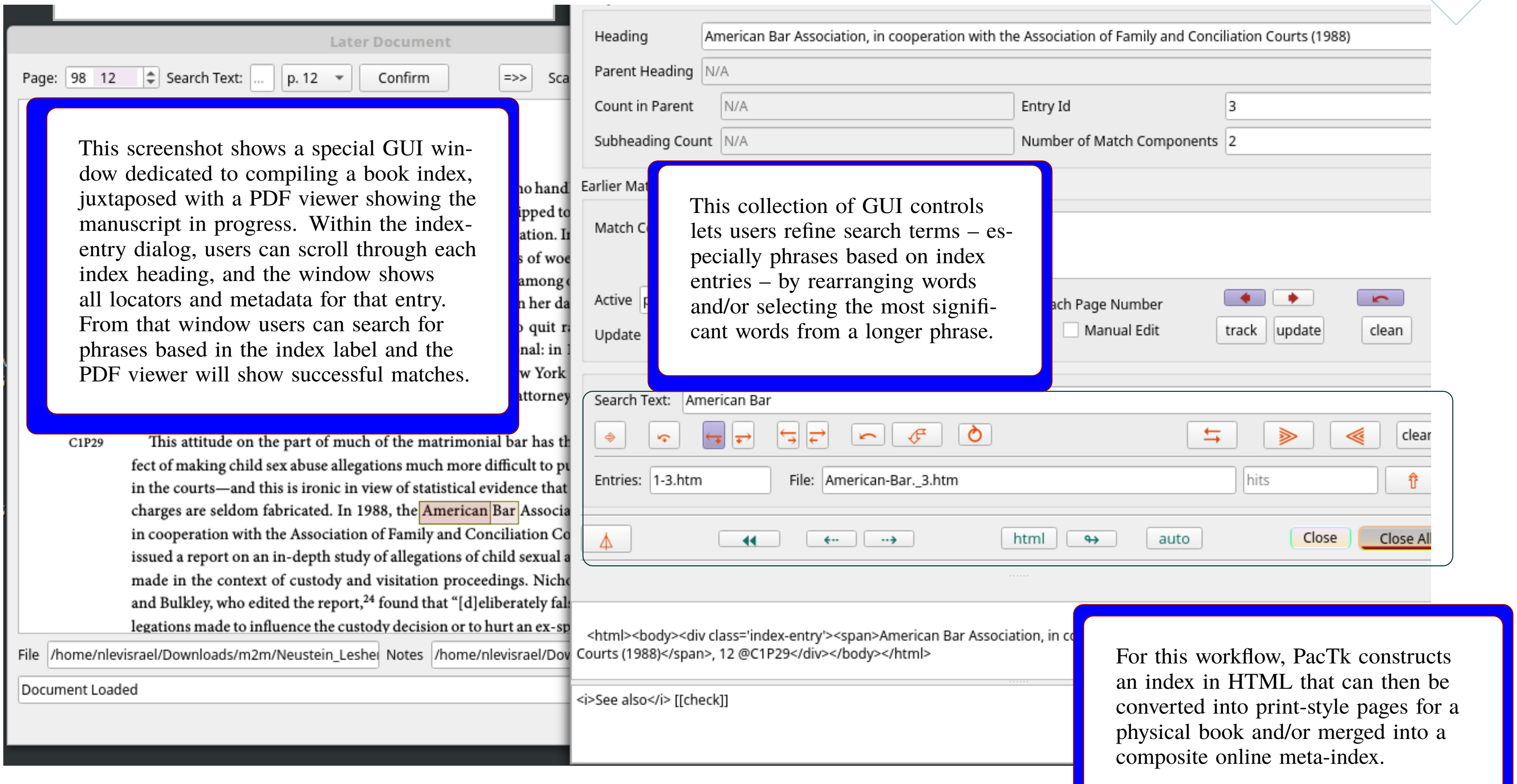


Figure 1: GUI functionality for compiling and curating indexes

- **Working with Research Data and Simulations** When publications are associated with data sets, code, or multimedia presentations, those supplemental materials should be indexed alongside the text manuscripts. This allows search engines to query all parts of a Research Object, not just documents. Equally important, indexing full Research Object allows publications to be cross-referenced with data, code, and/or multimedia.

  To see how multi-media cross-references might be used, suppose a publication includes a graphic taken from a video frame. Naturally, some readers will want to watch the entire video. Cross-referencing between that document-location and the video file would allow PDF viewers to play the video in a separate window (either from the beginning or from shortly before the still shot that was captured). Moreover, suppose the video shows a specific geographic location — for instance, it addresses environmental effects of climate change. By geotagging individual frames, the video player could open a digital map centered at the currently visible GIS coordinates. If the publication's Research Object includes GIS data sets, icons for dataset locations could then be superimposed on the base map.

  Alternatively, suppose a video is derived from a computer simulation — perhaps an analysis of how global warming will affect ecosystems, assuming different rates of greenhouse gas reduction (as input parameters). In addition to watching the video, readers might want to examine the simulation directly: modify the initial parameters or study the code implementing iteration updates. Here, multi-media cross-references could help a PDF viewer open multiple secondary windows, including a code preview and a dedicated window for the simulation graphics.

- **Digital Humanities** Executable Research Objects — combining text, code, and data into a single package — are identified with natural science, but similar programming techniques apply to the humanities as well. For instance, historical documents are themselves objects of study or assets to scholarship (e.g., collected papers). Archiving is more complex in this context because digital twins — whether as images, transcribed to text, or both — typically demand more detailed markup structure (for instance, transcription ambiguities are recorded, or scribbled marginal notes notated alongside the corresponding text). Or, consider a linguistic article that contains a list of sample sentences/passages, intended to demonstrate specific claims (about parsing structure, conceptualization, etc.) or to serve as a basis for subsequent analysis. Such samples (obtained from written corpora/audio records or simply invented by authors) are commonly typeset in a visually distinct style: usually printed with id numbers, contextual comments when needed, and sometimes special symbols representing phonology, agglutination, word order, morphosyntactic patterns, syntactic categories, and so forth.

  Because this mode of presentation is so common, researchers could benefit repositories of such examples, as a special kind of language corpus. That is, linguistics papers could be scanned for listed samples that get extracted into a common format, to be included in a multi-publication database. Interested readers could then browse the samples to find articles that address specific syntactic or semantic topics, particularly if each sample is classified according to the linguistic subject they exemplify (or serve thereto as a basis for analysis). Moreover, it is not uncommon for the same case study to be analyzed by multiple linguists, potentially creating interesting contrasts in theories and paradigms. The whole process would be accelerated if authors employ a common format (e.g., distinct XML tags) to notate the examples; but the full extracting and database persistence logic would need specialized programming.

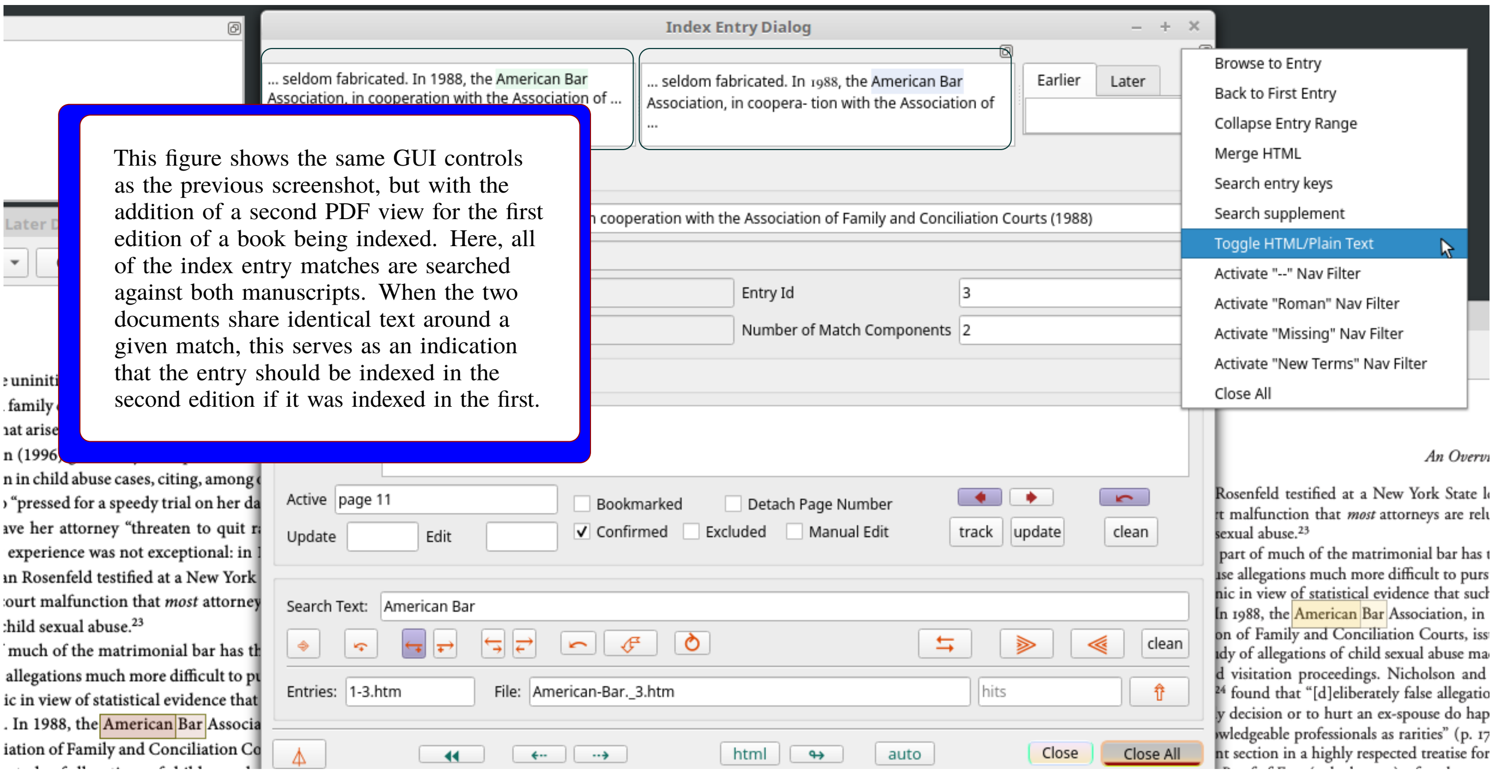


Figure 2: Compiling an index for the second edition of a book

- **Interoperating Publication Viewers with Science Applications** Graphics and data visualization provided by science applications can augment the functionality and interactive features of document viewers. Consider an article about biochemistry whose text content includes compounds identified either by a common name or a chemical formula. Assuming these special terms are ended in a structured format (e.g., special **XML** tags), chemical names/formulae can be associated with molecular-structure files in formats such as **PDB** (Protein Data Bank) or **CML** (Chemical Markup Language). Those files may then be opened via applications dedicated to visualizing and analyzing chemical structures in three dimensions (showing positions of atoms, atomic groups/arrangements, bonding strength, sub- and inter-molecular forces, and so forth). A similar example might be a chemistry textbook, with custom classroom software.

  For this kind of context, special programming is needed to extract textual data about chemical names and formulae, and potentially to search for molecular files via science **APIs**. Furthermore — in the hopes of best leveraging **3D** visualization capabilities — the **PDF** viewer has to interoperate with science applications, either through a common message-passing protocol or by embedding the entire **PDF** code base itself as a plugin or extension. Ideally, one or more molecular visualization applications themselves can be modified to work directly with these special **PDF** components, because — once **3D** files are loaded into the software — the specific chemical compound being studied, and its properties, could be cross-referenced to locations in the original research manuscript (or, analogously, the students' textbooks). Similar points could be made for interop between manuscript views and multimedia content of many kinds: digital map displays, videos (which may in turn be synced with digital maps by geotagging individual frames), simultations, image series, and so on.

▷ Indexing Digital/Multimedia Repositories

Within Executable Research Objects, text documents are paired with data, code, graphics, and/or multimedia resources. **PacTk** extends its meta-index protocol to incorporate these variegated asset types. Index code sources and raw data files through a framework oriented also to text manuscripts — rather than traditional pre-search ingesting associated with Content Management Systems (**CMS**) — has numerous benefits: • indexable terms may appear in many contexts,[1] not all of which are covered by **CMS** indexing; • indexes should be used not only for searching and external documentation but also incorporated into **GUI** functionality directly;[2] and cross-references between text and non-text content should incorporate such details as individual sentences and **PDF** page coordinates, which require rigorous text encoding.

Executable Research Objects are often associated with "grid" technology. In general, a grid is designed with the assumption that users will execute software packages which are accessible via the grid, and that their primary reason for doing so is participating in large, decentralized computations, or because the software encapsulates novel research. Although originally focused on remote code-execution on special-purpose computers, more recent use cases are more rigorously aligned with **FAIR** principles: academic papers often present research methods and theoretical models in abstract, mathematical terms. Observing visuals, and experimenting with code parameters,

---

1. Data types and their fields/members; scales and units of measurement; coordinate axes; dropdown combo boxes; context menus; tabular column labels; user actions available vis-à-vis specific **GUI** elements.
2. For instance, the executable portion of a Research Object could be run in a special "assist" mode where supplemental components illustrate how to use different **GUI** windows

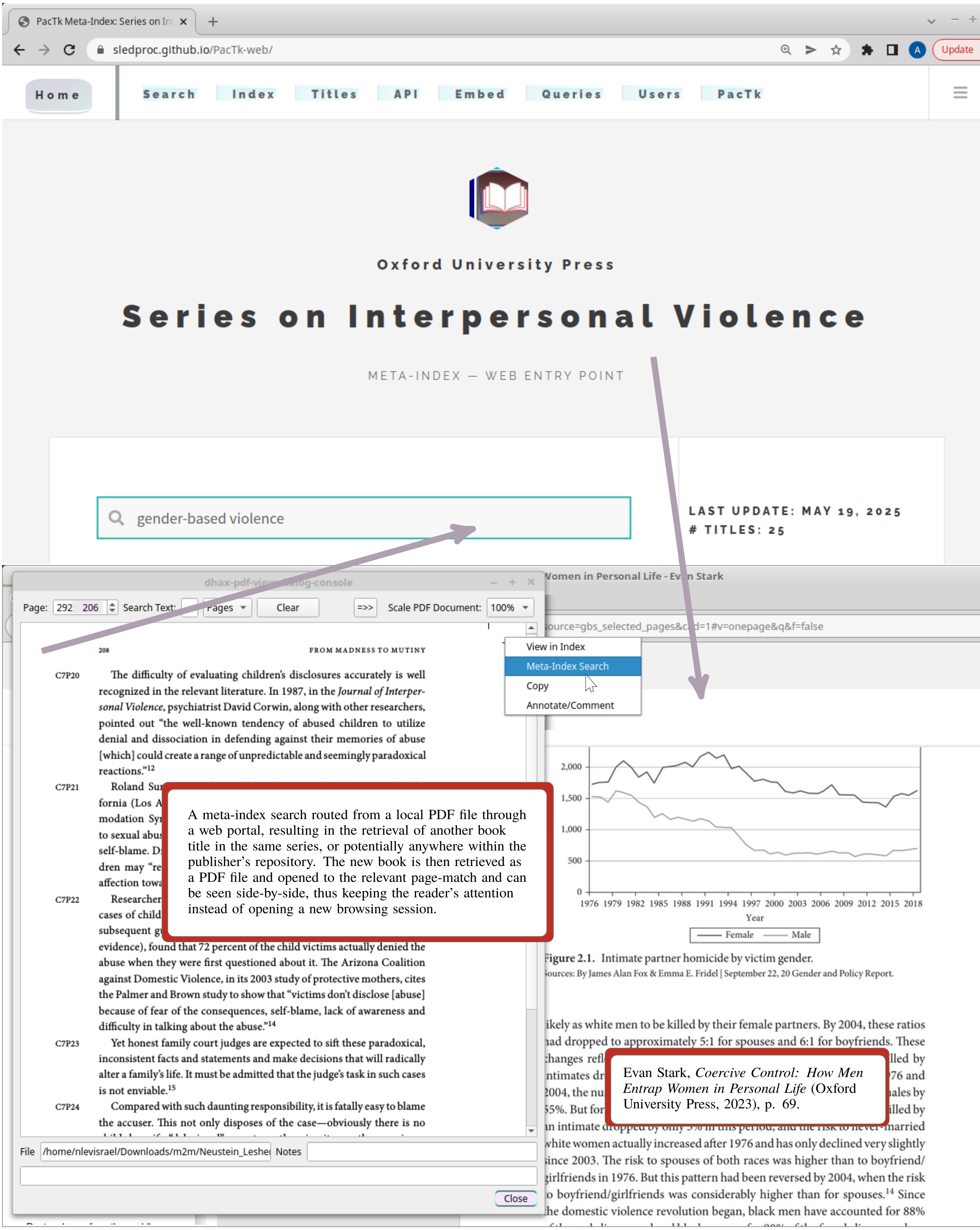


Figure 3: Meta-index search through a web portal

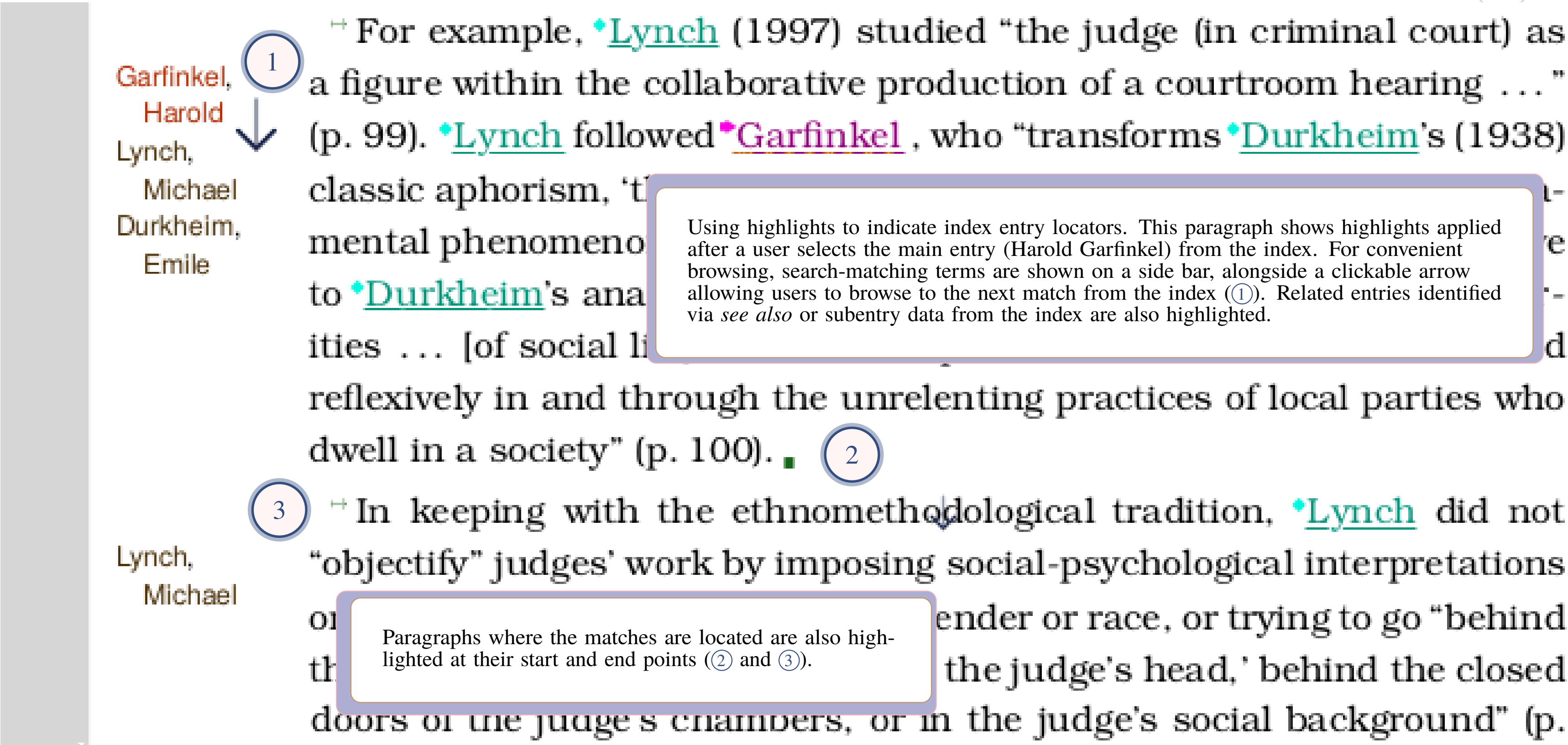


Figure 4: Index Search in a PDF Front-End Context

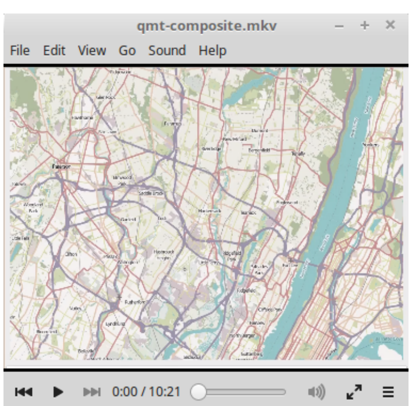
can help make the underlying research more understandable. There is pedagogical value in running and examining simulation and/or analytic code, that becomes accentuated insofar as code is developed with explicit conceptual semantics (pre- and post-condition declarations, Design by Contract, unit-decorated types, and so on). These code annotations may then become a basis for text-integrated indexing.

With this in mind, we can formulate several core principles of "**FAIR**" grid computing, including: • code packages hosted on a grid should be designed with as few mandatory external dependencies as possible; • where a given package has software or hardware requirements that are difficult to satisfy, a simplified version should be made available that would at least allow some provisional learning and evaluation of published results; • every effort should be made to ensure that the software executes in a usable fashion even in the absence of dependencies or preconditions that held true for those developing the code; • grid collaborations should offer replication-focused coding tools; and • there should be a clear set of actions, with as few steps as possible, allowing someone newly acquainted with a grid-hosted package — in particular, someone who learns about the code from a technical publication — to obtain and execute the relevant code. Researchers, especially, should be able to build and use the software in parallel with figures and text discussions featured in document manuscripts.

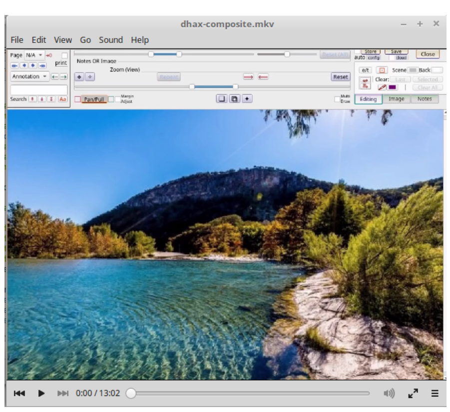
In general, then, publishers cannot assume that **FAIR** transparency is achieved simply by sharing raw data files. Instead, open-access materials should wherever possible include plugins or source-code extensions (developed individually for one manuscript; or collectively for a book series, journal title, or document repository) targeting science applications with requisite capabilities to view/analyze domain-specific file types. Similar comments apply to humanities environments, such as front-ends for studying linguistics corpora and/or parse-structures; sociological/economic statistics; or graphics tools for digital humanities. **PacTk** can help in this context because a single **PacTk** plugin may serve as middleware on top of which extensions could operate that are individualized to specific publications/manuscripts. This can be more efficient than building components against applications' native plugin mechanisms directly.

▷ Video Links

For more information, please see our software demo videos, which show software components that could be integrated into dataset applications and **PDF** viewers:



GIS



360° photography

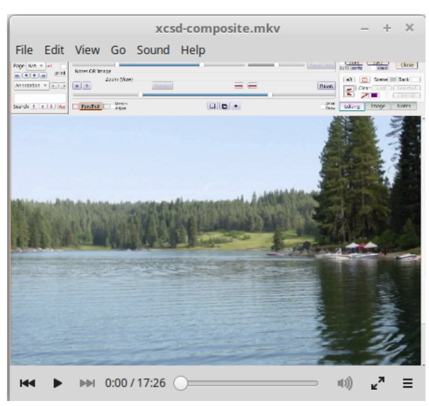


Image Processing

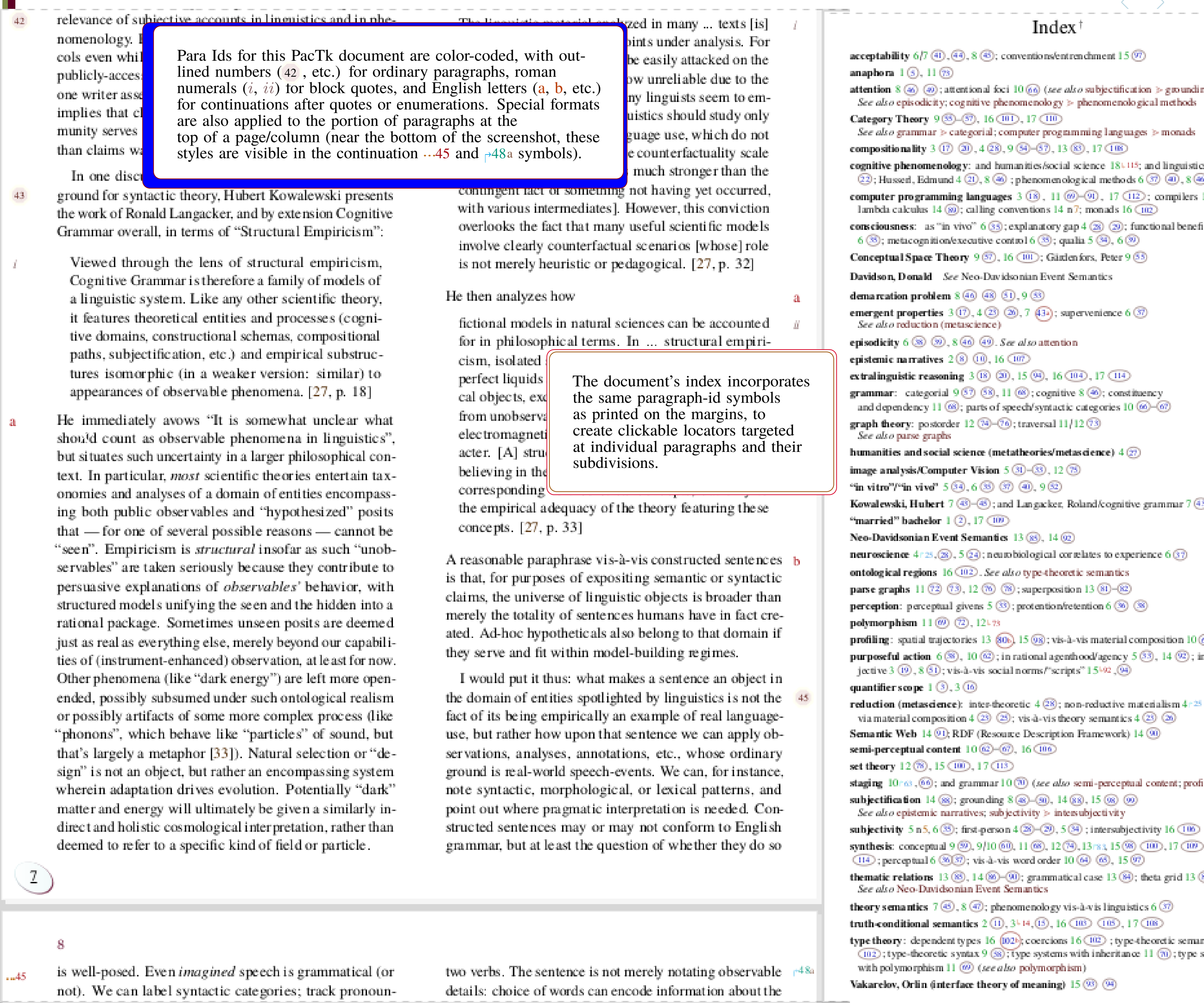


Figure 5: Subdivision formats for Paragraph IDs

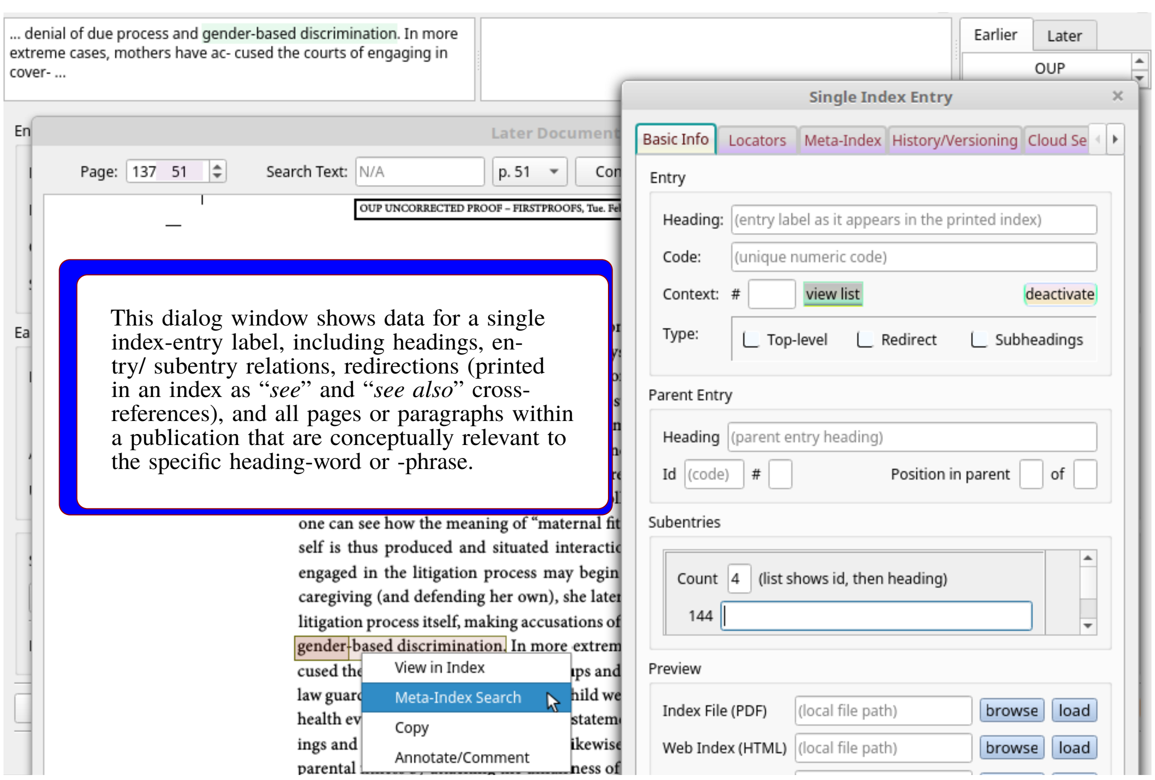


Figure 6: An index entry dialog window (and context menu for single and meta-index views)

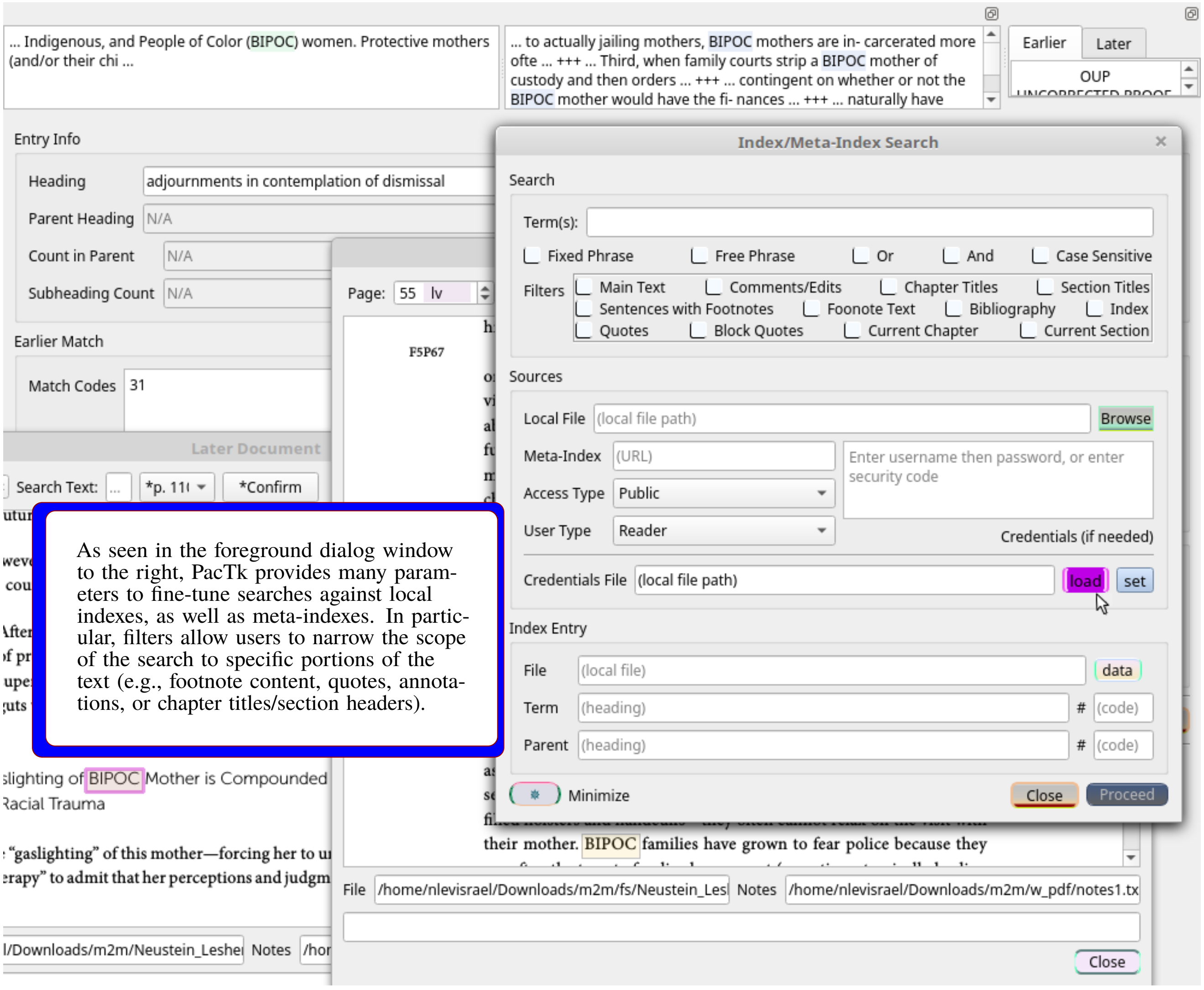


Figure 7: Dialog window for local and meta-index searches

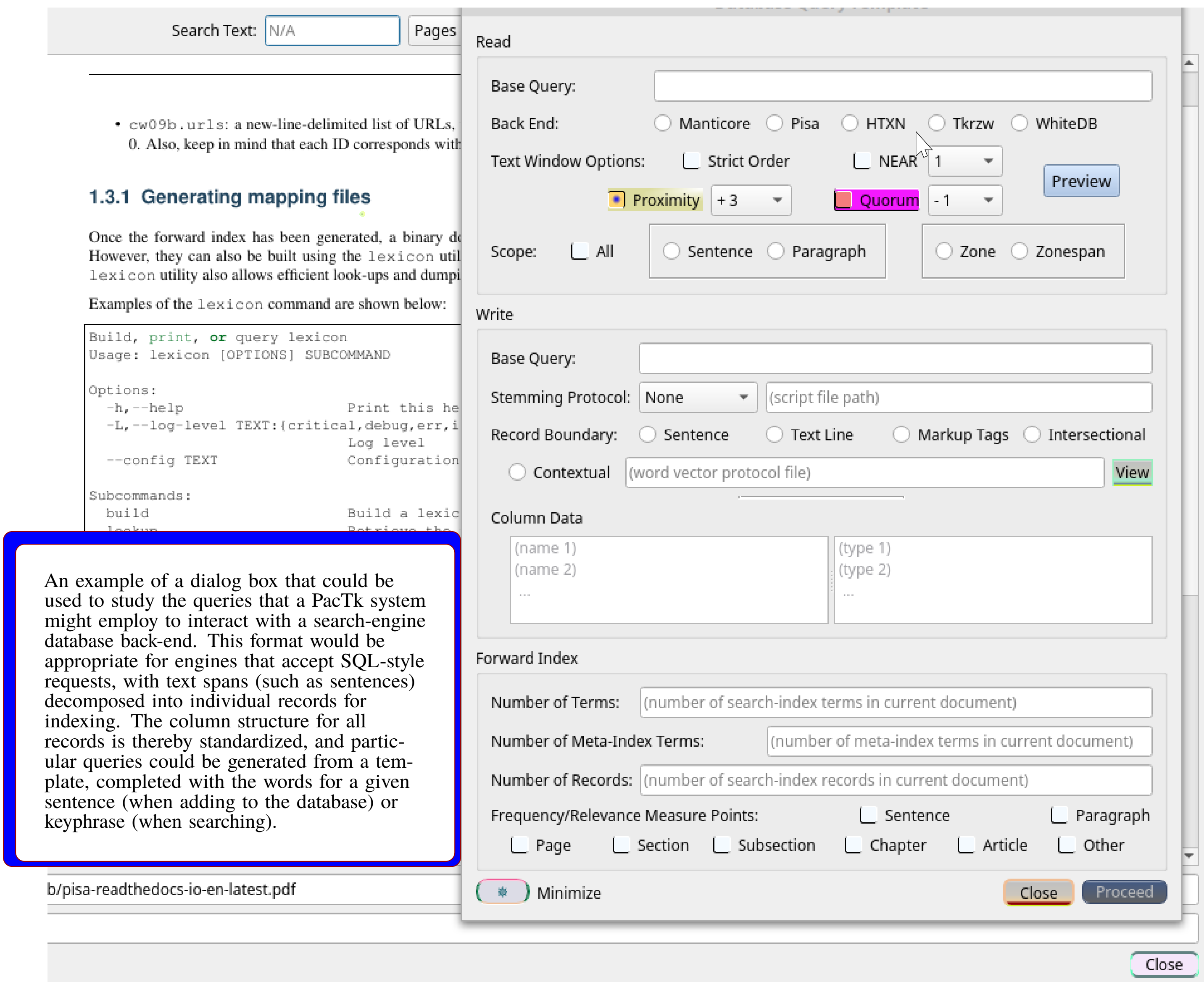


Figure 8: Configuring default back-end queries

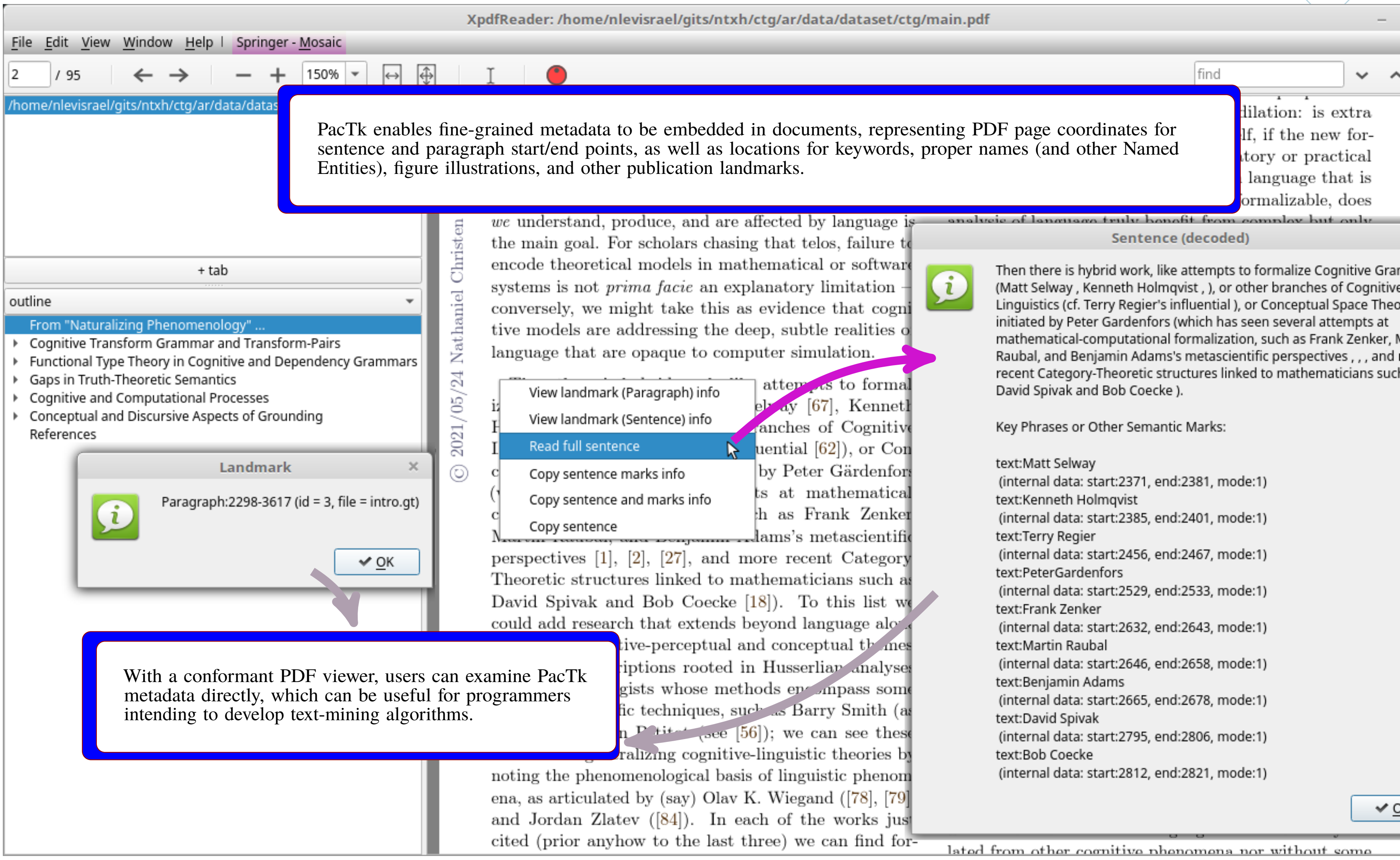


Figure 9: Enhanced GUI capabilities enabled by machine-readable text encoding

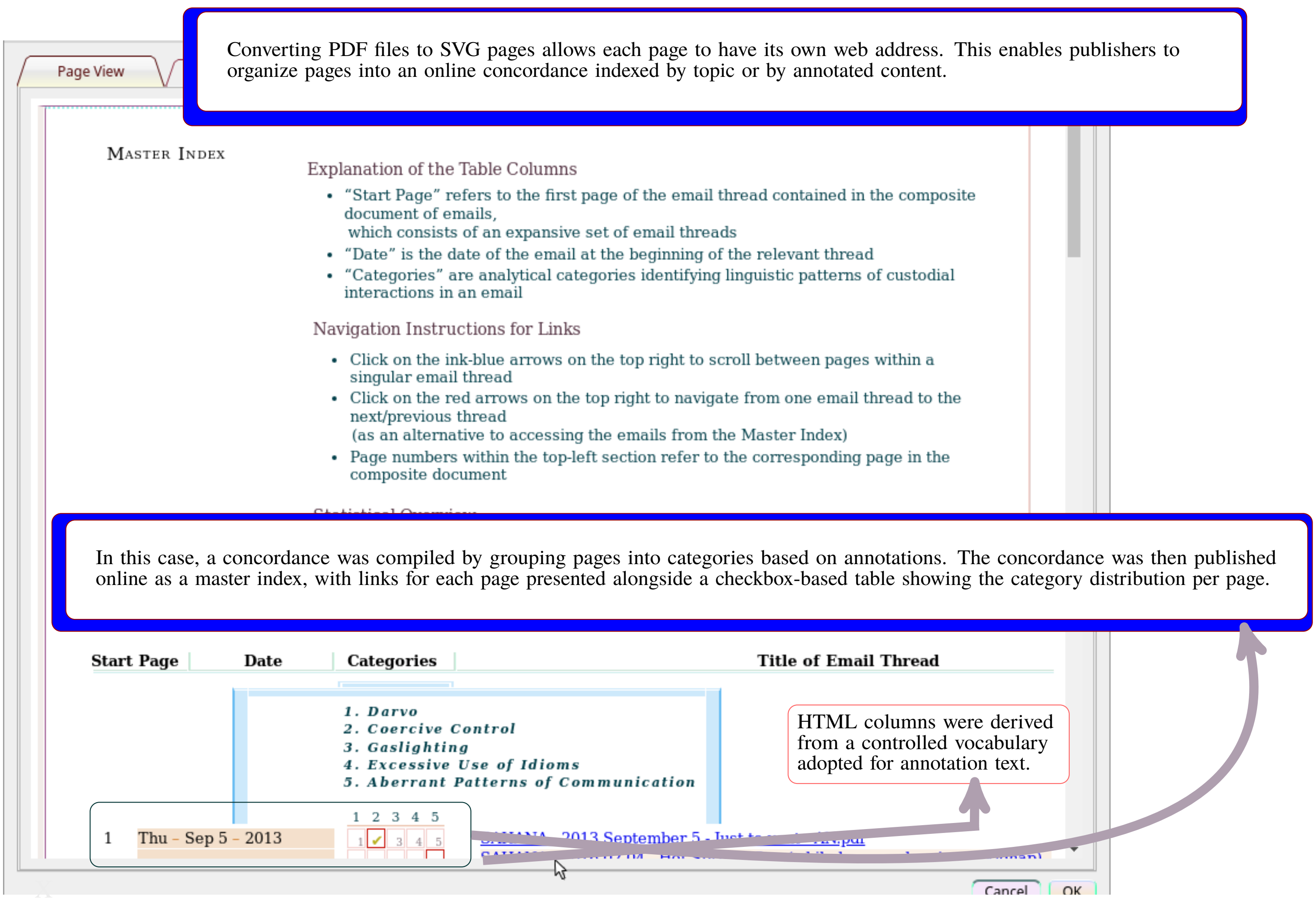


Figure 10: A category-based web-accessible index table for SVG pages